AutoML CI/CD/CT Capstone Project Final Report

Sepehr Heydarian, Archer Liu, Elshaday Yoseph, Tien Nguyen

Table of contents

1 Executive Summary

This project addresses a key bottleneck in our partner's workflow by developing an automated and modular pipeline for end-to-end model improvement. Our partner, Bayes Studio, develops AI-driven wildfire detection systems using edge-deployed camera devices. Their existing workflow consists of manual labeling, model training, and compression, which slows down the process of pre-labeling image data and updating models in a timely manner.

Our pipeline processes incoming data through a semi-automated labeling system that combines predictions from two models (YOLOv8 and Grounding DINO), with human-in-the-loop validation for uncertain cases. Labeled images then undergo data augmentation to increase dataset diversity. The pipeline fine-tunes a YOLOv8 base model on this new dataset and applies knowledge distillation and quantization to produce smaller, deployable models for edge devices.

Key outputs include a labeled dataset, fine-tuned and compressed models, and versioned metadata to support traceability and reproducibility. Although the system currently runs locally, it provides a strong foundation for scalable wildfire detection workflows.

Overall, this solution streamlines model updates and prepares models for real-time deployment on edge devices in wildfire detection scenarios.

2 Introduction

Wildfires represent a significant and growing environmental threat, with the potential to cause widespread ecological and economic damage. Defence Research and Development Canada (2022) emphasizes the importance of early detection in mitigating the impact and spread of these events.

Our Capstone partner, Bayes Studio, is a Vancouver-based startup developing advanced AI-driven tools for environmental monitoring and wildfire detection. They deploy specialized camera systems equipped with lightweight models that operate as edge devices, allowing for real-time and on-site inference.

At the outset of the project, Bayes Studio's workflow for updating their wildfire detection models involved several manual steps: labeling new environmental imagery, fine-tuning a base object detection model, and compressing it for edge deployment. This process is time-consuming and constrained their ability to quickly respond to changing conditions in wildfire imagery.

To address these challenges, we developed an automated and modular machine learning pipeline that supports continuous updates to wildfire detection models. The system is designed to handle incoming image data, generate high-quality object labels, retrain detection models, and compress them for edge deployment—with minimal manual effort.

The pipeline begins with a semi-automated labeling system that uses predictions from YOLOv8 and Grounding DINO. When the models agree, their outputs are accepted automatically; when they disagree, the images are routed to a human-in-the-loop review process using Label Studio. The labeled images are then augmented to improve diversity and robustness. A YOLOv8 model is fine-tuned on the resulting dataset, and the trained model is compressed through knowledge distillation and quantization to meet edge device constraints.

This system reduces labeling time, enables regular model updates, and improves deployment readiness. It provides Bayes Studio with a scalable and reproducible approach for fine-tuning and compressing models to edge devices. This enables timely and accurate wildfire detection directly on edge devices, supporting early intervention in high-risk environments.

3 Dataset

To develop and evaluate the pipeline, we used a dataset provided by Bayes Studio consisting of environmental imagery and corresponding object labels. The images were in standard formats such as `.jpg`, accompanied by YOLO-formatted `.txt` label files. Each label file corresponds to a single image and contains one line per object instance, with the format: `class_id x_center y_center width height`, where all bounding box values are normalized. For example:

```
1 0.757593 0.34663 0.06101 0.3655
```

This line indicates an object of class ID 1 with a bounding box centered at (0.757593, 0.34663) and a normalized width and height of 0.06101 and 0.3655, respectively.

The object detection task focuses on five wildfire-relevant categories:
1. Fire
2. Smoke
3. Lightning
4. Vehicle
5. Person

These categories were selected for their importance in early wildfire detection and response scenarios.

While the pipeline is designed to ingest raw, unlabeled images as input, this labeled dataset played a key role in building the initial components, testing functionality, and validating performance. The partner expects approximately 500 new images per month, which the pipeline is designed to process through labeling, training, and deployment stages.

4 Data Science Methods and Product

Our AutoML pipeline is designed for wildfire object detection, emphasizing automation, configurability, and efficiency. The pipeline operates in a continuous loop, processing new data and improving model performance through sequential stages of labeling, training, distillation, and quantization.

The pipeline consists of the following key stages:

1. **Automated Labeling and Matching**
   Object labels are automatically generated using two models: YOLOv8 and Grounding DINO. If both models agree, the prediction is accepted. Disagreements are flagged and routed for manual review.
2. **Human-in-the-Loop Review**
   Discrepant cases are corrected through a Label Studio interface. This ensures no mismatches are discarded, resulting in a high-quality training dataset.
3. **Data Augmentation**
   The labeled dataset is expanded using transformations such as flipping, rotation, brightness changes, and noise addition. This step increases data diversity and helps prevent overfitting.
4. **Model Training**
   A YOLOv8 model is trained on the combined labeled and augmented dataset. Training is controlled via a configuration file that supports reproducible experiments and CPU/GPU flexibility.
5. **Model Distillation**
   The trained model is compressed into a lightweight version using knowledge distillation. This improves inference speed while preserving detection accuracy.
6. **Model Quantization**
   The distilled model is further quantized to reduce size and enable deployment on edge devices with limited resources.

Key features of the pipeline include:

- **Automated**: From labeling to training and compression, everything flows with minimal manual effort.
- **Continuous**: Updated models help label data in future runs, improving over time.
- **Dynamic**: Easy to plug in new models or modules as requirements change.
- **Efficient**: Greatly reduces time across key stages including labeling, fine-tuning, and compression, resulting in faster end-to-end model iteration cycles.

To visually summarize this workflow, Figure 1 provides an overview of the complete pipeline architecture.

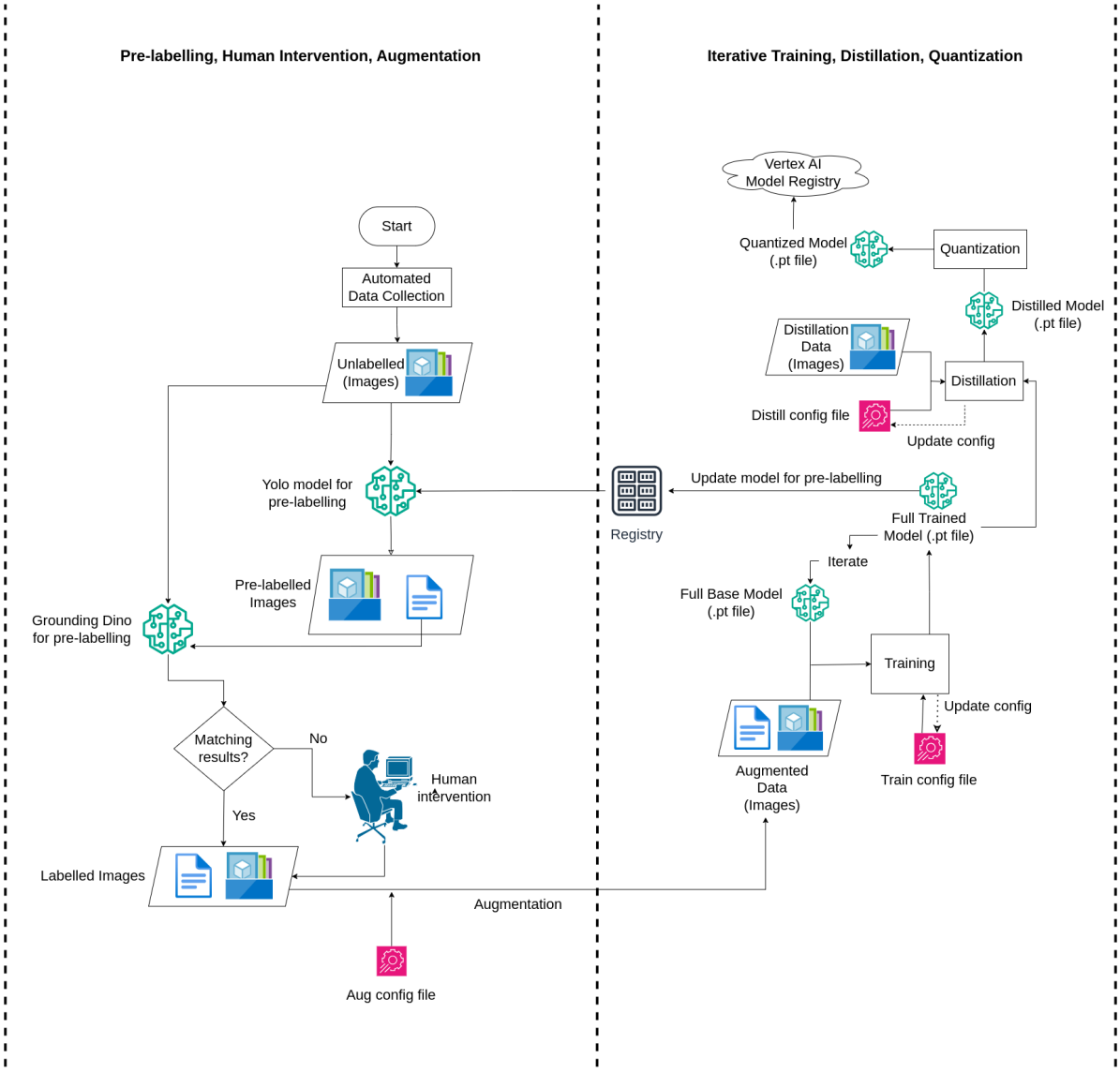

Figure 1: End-to-End Pipeline Overview

Each of these components is discussed in more detail in the following sections.

4.1 Automated Labeling and Matching

Manual annotation is time-consuming and prone to inconsistencies. To streamline this process, we employ a dual-model pre-labeling strategy using YOLOv8 (Jocher, Chaurasia, and Qiu 2023) and Grounding DINO (Liu et al. 2023). This setup leverages model agreement to reduce manual labeling needs while preserving uncertain cases for human review. The goal is to automatically generate accurate labels with minimal intervention.

Figure 2 below illustrates this labeling and matching process.

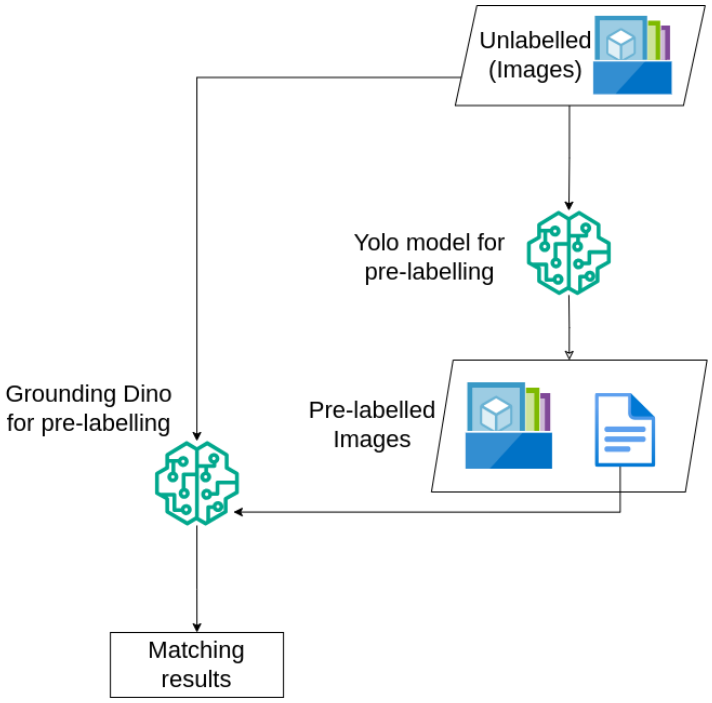

Figure 2: Labeling and Matching Workflow

The labeling workflow proceeds as follows:

1. **Input**
    - Unlabeled images are passed through the pipeline.
    - Two object detection models are used:
        - **YOLOv8**: Trained specifically on five wildfire-related categories – Fire, Smoke, Lightning, Human, and Vehicle.
        - **Grounding DINO**: Uses the same categories as prompts to perform open-vocabulary detection.

2. **Workflow Steps**

   - **Step 1:** YOLOv8 predicts bounding boxes and class labels.
   - **Step 2:** Grounding DINO makes independent predictions using the same category prompts.
   - **Step 3:** A matching script compares predictions using an Intersection over Union (IoU) threshold:
     - If predictions align, the image is automatically labeled.
     - If predictions disagree, the image is flagged for human review.
   - **Step 4:** Confirmed matches are saved in a standardized format for downstream use.

3. **Output**

   - A curated dataset of confidently labeled images.
   - Uncertain cases routed for human-in-the-loop review.
   - Each labeled file contains bounding boxes, class names, and confidence scores.

4. **Impact**

   This automated labeling workflow reduces reliance on full manual annotation, ensures consistent labeling across large batches, and accelerates dataset preparation while maintaining label accuracy.

4.2 Human-in-the-loop

A high-performing model relies on a high-quality labeled dataset While YOLOv8 and GroundingDINO agree on many predictions, there are still mismatches that need correction. Instead of discarding these potentially valuable samples, we introduce a human-in-the-loop workflow to verify and fix labeling errors with Label Studio (Heartex 2021). This process improves data quality with minimal manual effort, streamlining the correction process and reducing the need for manual file editing.

1. **Input**

   - JSON files with incorrect or uncertain predictions from YOLO, containing labels, bounding box coordinates, and confidence score
   - Corresponding image files referenced in those predictions

2. **Workflow Steps**

   Figure 3 summarizes the overall human-in-the-loop workflow.

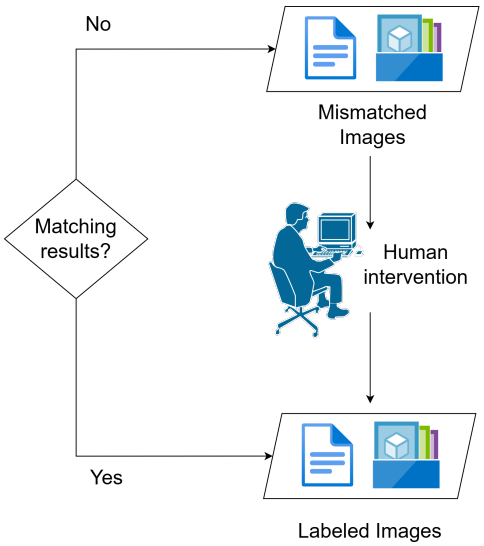

Figure 3: Human-in-the-loop workflow diagram

- **Step 1:** Place mismatched prediction files in the pending directory
- **Step 2:** Run the script to convert them into Label Studio tasks using images from the input directory
- **Step 3:** Review and fix labels visually in the intuitive Label Studio interface as shown in Figure 4

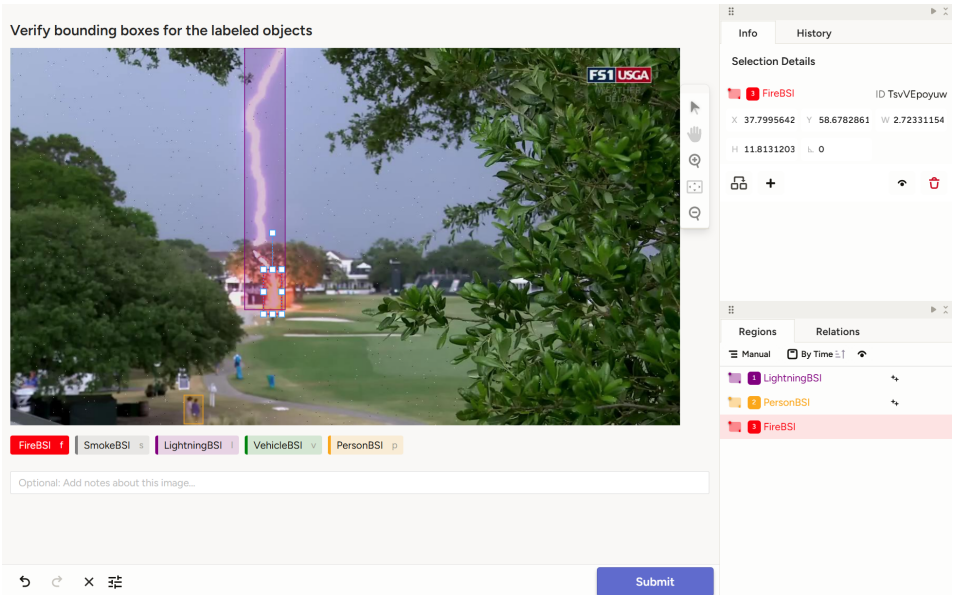

Figure 4: Label Studio Interface for Human Review

- **Step 4:** Export the reviewed results to label studio results directory
- **Step 5:** Reviewed prediction files are moved from pending directory to labeled directory, available for the training phase

The script automatically tracks review status throughout the process.

3. **Output**

   - Corrected label files (JSON), ready for training
   - The file names of Label Studio outputs are versioned with timestamps automatically for easy tracking

4. **Impact**

   The human-in-the-loop workflow makes better use of mismatched data. Reviewers can easily correct labels through an intuitive interface, without editing raw files manually. The reviewed results are saved in a consistent format, ready for training. Overall, this workflow improves the quality of the training dataset, allowing the model to learn from a wider range of examples.

4.3 Data Augmentation

A single scene can look very different depending on external conditions such as lighting, weather, occlusion, and the angle or position of the camera. To help the model generalize better to these real-world variations, we apply data augmentation techniques to artificially increase the diversity of the training dataset. This process improves robustness and reduces the risk of overfitting, especially in low-data or imbalanced class scenarios.

In this project, the partner specifically requested that augmentations be applied prior to training, rather than using in-training augmentation. This decision was motivated by the need to minimize computational overhead and training time.

1. **Input**

The augmentation process uses labeled images along with their corresponding YOLO-formatted annotation files. These images are sourced from the matched outputs of the pre-labeling pipeline and are stored in the `labeled` directory. Augmentation behavior is controlled via the `augmentation_config.json` file, which allows customization of parameters such as the number of augmentations per image, transformation probabilities, and limits for effects like blur intensity.

2. **Workflow Steps**

- **Step 1**: Load each image and its corresponding annotation.
- **Step 2**: For each image, generate a fixed number of augmented versions (e.g., 3).

- **Step 3**: Apply a set of transformations using the `Albumentations` library, including:
  - Horizontal flips
  - Brightness and contrast adjustments
  - Hue and saturation shifts
  - Gaussian blur (with a configurable limit)
  - Gaussian noise
  - Rotations
  - Conversion to grayscale
- **Step 4**: Update the bounding boxes to reflect the applied transformations. This is handled automatically by the `Albumentations` library. Images without any objects of interest are excluded from augmentation.
- **Step 5**: Save the augmented images along with their updated annotations.

3. **Output**

The result is a larger and more varied labeled dataset prepared for training. This includes both the augmented images with associated annotation files and the original non-augmented images. Images without any predicted or labeled objects are stored separately in the `no_prediction_images` folder.

4. **Impact**

By augmenting the data prior to training, the model is exposed to a wider range of visual conditions without requiring additional manual data collection or labeling. This helps reduce overfitting and improves generalization performance, particularly under different lighting conditions, weather variations, and camera viewpoints.

4.4 Model Training

Motivation

Once labeled and augmented data is available, the detection model must be re-trained to incorporate new information. Continuous re-training enables the model to adapt to evolving wildfire conditions and the characteristics of newly collected data. The goal of this stage is to automate the fine-tuning of a base YOLOv8 model using the most recent dataset, minimizing the need for manual effort.

1. **Input**

The training stage uses the full set of labeled images, including both augmented and non-augmented examples. Images without any labeled objects are also included as true negatives. The model to be fine-tuned is selected from the model registry. By default, this is the most recently updated model yolo model from previous iteration of the pipeline, though users can also specify a custom path or fall back to an initial base model such as `nano_trained_model.pt`.

Training hyperparameters including settings like the number of epochs, learning rate, and input image size are defined in the `train_config.json` file. This configuration supports any training argument compatible with the YOLOv8 framework.

2. **Workflow Steps**

- **Step 1**: Load the training configuration and identify the most recent base model.
- **Step 2**: Organize the data into training, validation, and test splits.
- **Step 3**: Fine-tune the base YOLOv8 model using the prepared data and configuration settings.
  - Training is conducted using the Ultralytics YOLOv8 training API.
  - Performance metrics are logged during training.
- **Step 4**: Save the updated model in the model registry using a timestamped filename.
- **Step 5**: Store metadata, including training date, configuration settings, and performance results, in a corresponding metadata file.

3. **Output**

The training stage produces a fine-tuned YOLOv8 model that reflects the latest available data. The model is saved with a filename that follows the pattern `[YYYY-MM-DD]_[HH_MM]_updated_yolo.pt`, and is stored in the model registry. A metadata file with the same name is generated with records of the training hyperparameters to reproducibility. The metadata file also includes the following test metrics: `map_50`, `map_75`, `map_50_95`. Additional output includes evaluation artifacts such as the confusion matrix, F1 curve, precision-recall curve, and `results.csv` summary file generated by the YOLO training API.

4. **Impact**

This process enables regular fine-tuning as new data becomes available, helping ensure that the detection model remains accurate and up-to-date. The modular structure also allows users to experiment with different hyperparameters or revisit older models for retraining as needed. Over time, this approach supports continual improvement in model performance and maintainability.

4.5 Knowledge Distillation

Large object detection models, while accurate, demand significant computational resources, limiting real-time wildfire detection on edge devices. Knowledge distillation addresses this by transferring expertise from a larger **teacher model** to a smaller, more efficient **student model**. This enables faster inference while maintaining accuracy, making models suitable for resource-constrained environments.

1. **Input**

   - **Teacher model:** The latest fine-tuned, high-performing YOLOv8 model for wildfire detection, larger than YOLOv8 nano.
   - **Student model:** A smaller YOLOv8n architecture initialized with pre-trained weights.
   - **Distillation dataset:** Contains 5 classes (FireBSI, LightningBSI, PersonBSI, SmokeBSI, VehicleBSI).
   - **Configuration file (`distillation_config.yaml`):** Specifies model paths, dataset path, and distillation parameters.

2. **Workflow Steps**

   The distillation workflow ensures effective knowledge transfer:

   - **Step 1:** Load and prepare both teacher and student models.
   - **Step 2:** Freeze the first 10 layers (backbone) of the student model to preserve pre-trained features.
   - **Step 3:** Train the student model using a balanced **combined loss function**:
     - **Detection loss:** Direct supervision from ground truth labels.
     - **Distillation loss:** Facilitates knowledge transfer from the teacher, comprising:
       * **Bounding box distillation:** Ensures accurate localization.
       * **Classification distillation:** Preserves class prediction.
   - **Step 4:** The student model is trained/distilled in a loop with regular saving and logging. **Early Stopping** triggers if validation performance does not improve after a set number of epochs, exiting the loop and preserving the best weights.
   - **Step 5:** Save the final best weights of the distilled model for downstream tasks and deployment.

   Early Stopping's validation criteria (`fitness`) combine these 4 metrics:

   - **Precision (P):** Accuracy of detected objects (correct detections).
   - **Recall (R):** Model's ability to identify all object instances.
   - **mAP50:** Mean average precision at 0.50 IoU, measuring accuracy for "easy" detections.

- **mAP50-95:** Average mAP across IoU thresholds (0.50 to 0.95), offering comprehensive performance view.

3. **Output**

   - Distilled YOLOv8n model, reduced in size and complexity, saved in a user-specified directory.
   - Training logs and metrics for performance analysis.
   - Checkpoint files for training resumption.

4. **Impact**

   Knowledge distillation offers a practical solution for deploying wildfire detection models by significantly reducing model size and computational load. While some performance drop is inherent, effective knowledge transfer minimizes this loss. The resulting model achieves faster inference, ideal for real-time edge device applications.

4.6 Quantization

Even after model distillation, running inference on edge devices can still be too slow or resource-intensive. To address this, quantization is used to reduce the size of the model and improve inference speed by lowering the numerical precision of model weights. This makes the model more efficient and suitable for real-time wildfire detection in the field.

1. **Input**

   - The distilled YOLO model from the previous distillation step of the pipeline
   - (Optional) labeled images and JSON annotations for calibration, required for IMX quantization

2. **Workflow Steps**

   - **Step 1:** Set quantization method in `quantize_config.json` (`"FP16"`, `"ONNX"`, or `"IMX"`)
   - **Step 2:** Based on method:
     - **FP16**: Converts weights to float16, fast and accurate, saved as `_fp16.pt`
     - **ONNX**: Converts to ONNX and applies dynamic quantization, saved as `_onnx_quantized.onnx`
     - **IMX**: Uses Ultralytics export with calibration data to create IMX-ready format (Linux only), saved as `_imx.onnx`
   - **Step 3:** Quantized model is saved in quantized model directory

3. **Output**

   - A smaller, faster model ready for deployment:

- ~50% size reduction with FP16, minimal accuracy drop
- ~75% size reduction with ONNX, slight accuracy tradeoff
- IMX-ready model for Sony Raspberry Pi AI Camera

4. **Impact**

   The quantized model is lightweight enough to run directly on edge devices that have limited computing power. This meets the partner's need to deploy the model on Sony Raspberry Pi AI Cameras for real-time wildfire detection. Additionally, there are multiple quantization options (FP16, ONNX, IMX) available, making it easy to switch methods based on different deployment needs in the future.

5 Conclusions and Recommendations

5.1 Problem Recap

Our partner needed a continuous pipeline to keep their machine learning models accurate and up to date as new image data comes in. Their existing process relied heavily on manual labeling, training, and deployment, which slowed down model updates and made it difficult to scale. A more automated and flexible solution is required to support timely updates and edge deployment.

5.2 Our Results

We developed a Python-based, modular pipeline that streamlines the entire workflow.

All options, paths, and process steps are set in centralized configuration files. The pipeline reads these files at runtime to decide which steps to run, which folders to use, and what parameters to apply. This makes the workflow reproducible, portable, and easy to update for new needs.

5.3 Limitations & Recommendations

While our pipeline meets the core requirements, there are some limitations that restrict the scalability of the pipeline:

- **Local-first design**: Running the pipeline locally limits deployment flexibility.
- **Scattered data and model files**: Data and model files are saved in local folders, which makes tracking progress and maintaining consistency more difficult.
- **No monitoring interface**: Without a monitoring interface, it's hard to observe pipeline behavior in real time.

Hence, we propose the following three recommendations for future development:

- **Add cloud support**: Integrating cloud storage and remote triggering to allow the pipeline to run in more flexible environments.
- **Integrate a lightweight database**: Using a small database (e.g., SQLite) to store metadata for images, labels, and model versions would improve organization, enable easier tracking, and support more complex queries and analysis. As an additional task, we have designed a simplified and scalable database schema to support the core components of our wildfire detection pipeline. The structure is designed to be future-proof and manageable by the partner team.
- **Develop a simple monitoring dashboard**: A lightweight dashboard would help users monitor pipeline runs, track model status, and show performance in real time.

References

Defence Research and Development Canada. 2022. "DRDC Tests Early Detection Wildfire Sensors." https://science.gc.ca/site/science/en/blogs/defence-and-security-science/drdc-tests-early-detection-wildfire-sensors.

Heartex. 2021. "Label Studio: Data Labeling Platform." https://labelstud.io/.

Jocher, Glenn, Ayush Chaurasia, and Jing Qiu. 2023. "Ultralytics YOLOv8." https://github.com/ultralytics/ultralytics.

Liu, Shilong, Zhaoyang Zhang, Yujie Lin, Tete Zhang, Chen Qian, Alan L Yuille, Xin Lu, Pengchuan Gao, and Jiuxiang Gu. 2023. "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection." *arXiv Preprint arXiv:2303.05499*. https://github.com/IDEA-Research/GroundingDINO.